Лабораторная работа №2. Параллельное вычисление определённого интеграла методом средних прямоугольников. Вариант №2.

1. Описание выполненной работы
   В работе реализованы последовательная и параллельная (OpenMP) версии метода средних прямоугольников для вычисления определённого интеграла. Параллельная версия использует директиву reduction для суммирования результатов. Проведены три эксперимента: влияние числа потоков при n = 10^8, влияние числа разбиений при 8 потоках и вычисление интеграла с заданной точностью ε = 10^{-9}.

2. Таблицы с результатами экспериментов.
   Эксперимент 1. Влияние количества потоков (n = 10^8):

Потоки	Время, с	Ускорение
1	0.460	0.993
2	0.419	1.259
4	0.298	1.849
8	0.235	2.634
16	0.188	2.457

   Эксперимент 2. Влияние числа разбиений (8 потоков):

n	Знвчение интеграла	Время, с
10^4	0.904524238601	0.002
10^5	0.904524237907	0.003
10^6	0.904524237900	0.005
10^7	0.904524237900	0.029
10^8	0.904524237900	0.263

   Эксперимент 3. Результат вычисления с заданной точностью (ε = 10^{-9}):

	n	Знвчение интеграла	Время, с
последоват.	16384	0.904524238161	0.0010
паралл.	16384	0.904524238161	<0.001

3. Вывод о проделанной работе.
   С ростом числа потоков время выполнения программы уменьшается, однако ускорение растёт нелинейно: наибольший прирост достигается при переходе от 1 к 8 потокам, а при 16 потоках ускорение снижается из-за ограниченного числа физических ядер процессора и накладных расходов OpenMP на создание потоков и синхронизацию. Число разбиений напрямую влияет на точность результата: при малых n значение интеграла заметно отличается от эталона, но уже при n порядка 10^6 оно сходится к точному значению 0.9045242379 с высокой точностью, а время выполнения при этом растёт

линейно, что соответствует сложности метода O(n). Для суммирования результатов по потокам в программе используется директива reduction, которая создаёт локальную копию суммы в каждом потоке и объединяет их после завершения цикла - это исключает гонку данных и обеспечивает почти линейное ускорение без блокировок. Альтернативные механизмы critical и atomic были бы медленнее, поскольку заставляли бы потоки ждать друг друга на каждой итерации. Таким образом, применение OpenMP с reduction позволяет эффективно распараллелить метод средних прямоугольников и получить значительный выигрыш во времени при больших n.